

Armchairs — CF-1800 (1525D)

AUTHORED BY: ARPIT RAJ • LNMIIT JAIPUR • OCTOBER 1, 2026

Problem Formulation

- 1 to n chairs, some occupied.
- i^{th} occupied, j^{th} not occupied.
 $(i \rightarrow j) \rightarrow \text{time} = |j - i|$
- Sequential operation, one by one.
- $a_i = 1$ & implies; occupied.
- **Goal:** Minimum minutes taken so that every seat initially occupied is free.



Fig 1: Original Problem Breakdown Notes

Eg 1:

1 0 0 1 0 0 1 $\rightarrow$ o/p = 3

Eg 2:

1 1 1 0 0 0 (Indices: 1 2 3 4 5 6)

- 1 $\rightarrow$ 4 (in 3 minutes)
- 2 $\rightarrow$ 5 (in 3 minutes)
- 3 $\rightarrow$ 6 (in 3 minutes)

$\rightarrow$ **9 minutes**

Basically 1, 2, 3 were marked unusable. 4, 5, 6 were usable.

Array Extraction

one = [1, 4, 7]

zero = [2, 3, 5, 6]

- 1 $\rightarrow$ 2 done
- 4 $\rightarrow$ use 3 or dont use it

Dynamic Programming State & Transitions

STATE DEFINITION

$dp[i][j]$ = cost to place first i people using first j empty seat.



Fig 2: DP Formulation Notes

Choice 1:

Don't use empty seat.

$$\rightarrow dp[i][j] = dp[i][j-1]$$

Choice 2:

Use it.

$$\rightarrow dp[i][j] = dp[i-1][j-1] + \text{abs}(\text{ones}[i-1] - \text{zeros}[j-1])$$

** for 0 indexing*

BASE CASE

$$\rightarrow dp[0][j] = 0$$

C++ Implementation

The code below optimally implements the described $O(n^2)$ Dynamic Programming approach matching the formulated state choices and base cases. It is strictly formatted as originally provided.

```
#include<bits/stdc++.h>
using namespace std;

int main(){

    int n;
    cin>>n;

    vector<int> ones;
    vector<int> zeros;

    for(int i=0;i<n;i++){
        int x;
        cin>>x;

        if(x==1){
            ones.push_back(i);
        }
        else{
            zeros.push_back(i);
        }
    }

    int m1=ones.size();
    int m2=zeros.size();

    vector<vector<int>> dp(m1+1,vector<int>(m2+1,INT_MAX));
    for(int j=0;j<=m2;j++){
        dp[0][j]=0;
    }

    for(int i=1;i<=m1;i++){
        for(int j=1;j<=m2;j++){

            dp[i][j]=dp[i][j-1];

            if(dp[i-1][j-1]!=INT_MAX){
                dp[i][j]=min(dp[i][j],dp[i-1][j-1]+abs(ones[i-1]-zeros[j-1]));
            }
        }
    }

    return dp[m1][m2];
}
```